

CSIT 314

Software Development Methodology

Name	UOW ID
Chew Chang Jun Casper	9181891
Nyan Lynn Htun	1104597
I'sa Hong Xi	1036026
Fahmi Azri	8335357
Wang Jia Wei Ivan	7770303
Oh Bei Rui	1149659
Eshwer S/O Gunaseelan	1148527

User stories as a Donee

1.	As a Donee, I want to filter campaigns by urgency tier (Critical / Nearly Funded / Ongoing) so that I can prioritize campaigns where my donation will have the most immediate impact.	<pre> classDiagram class UrgencyFilterPage { -selectedTier: UrgencyTier -campaignResults: List -sortOrder: String +selectUrgencyTier(tier) +displayTierBadge() +renderFilteredList() +showProgressIndicator() } class CampaignFilterController { -tierFilter: UrgencyTier -thresholdConfig: Map +classifyCampaigns() +filterByTier(): List +computeUrgencyScore() +sortByImpactPotential() +getThresholdConfig() } class Campaign { -campaignId: String -targetAmount: Float -raisedAmount: Float -deadline: Date -urgencyTier: UrgencyTier +getUrgencyTier() } class UrgencyTier { -tierId: String -label: String [Critical, Nearly Funded, Ongoing] -thresholdPct: Float } UrgencyFilterPage --> CampaignFilterController : uses CampaignFilterController --> Campaign : queries CampaignFilterController --> UrgencyTier : queries </pre>
2.	As a Donee, I want to view a campaign's verification badge and trust indicators (e.g. ID-verified FR, official documents uploaded) so that I can assess legitimacy before committing a donation.	<pre> classDiagram class CampaignTrustPage { -campaignId: String -trustScore: Float -isVerified: Boolean +displayVerifiedBadge() +showDocumentList() +expandTrustDetails() +renderTrustMeter(score) } class TrustVerificationCtrl { -campaignId: String +getVerificationRecord() +computeTrustScore() +checkDocumentStatus() +isIDVerified(): Boolean +getBadgeLevel(): String +flagUnverified() } class VerificationRecord { -recordId: String -campaignId: String -idVerified: Boolean -docsUploaded: List -verifiedAt: DateTime +getRecord(): VerifRecord } class TrustScore { -scoreId: String -campaignId: String -score: Float -badgeLevel: String +compute() / + save() } CampaignTrustPage --> TrustVerificationCtrl : uses TrustVerificationCtrl --> VerificationRecord : reads TrustVerificationCtrl --> TrustScore : writes </pre>
3.	As a Donee, I want to receive an in-app alert when a saved campaign is within 10% of its fundraising goal so that I can donate before the opportunity closes.	<pre> classDiagram class AlertNotificationPanel { -doneId: String -pendingAlerts: List +showAlertBanner() +displayGoalProgress() +navigateToCampaign() +dismissAlert(alertId) +enablePushNotif() } class GoalAlertController { -doneId: String -alertThreshold: Float +checkSavedCampaigns() +computeGapToGoal() +triggerAlert(campaign) +markAlertSent() +getPendingAlerts() } class FavouriteList { -favouriteId: String -doneId: String -campaignId: String +getByDonee(): List } class AlertRecord { -alertId: String -doneId: String -campaignId: String -sentAt: DateTime -isDismissed: Boolean +save() / + dismiss() } AlertNotificationPanel --> GoalAlertController : uses GoalAlertController --> FavouriteList : reads GoalAlertController --> AlertRecord : writes </pre>
4.	As a Donee, I want to set up a recurring monthly donation to a campaign I trust so that I can contribute consistently without repeating the checkout process each time.	<pre> classDiagram class RecurringDonationPage { -monthlyAmount: Float -billingDay: Integer -paymentMethod: String +setMonthlyAmount() +chooseBillingDay() +confirmRecurringPlan() +showNextBillingDate() +cancelPlan() } class RecurringDonationCtrl { -doneId: String -campaignId: String +createRecurringPlan() +scheduleNextPayment() +processMonthlyCharge() +pausePlan(planId) +cancelPlan(planId) +getActivePlans(): List } class RecurringPlan { -planId: String -doneId: String -campaignId: String -monthlyAmount: Float -billingDay: Integer -status: String } class Donation { -donationId: String -planId: String -amount: Float -isRecurring: Boolean +save() / + getByPlan() } RecurringDonationPage --> RecurringDonationCtrl : uses RecurringDonationCtrl --> RecurringPlan : creates RecurringDonationCtrl --> Donation : creates </pre>

5.	As a Donee, I want to split a single donation across multiple campaigns in one checkout session so that I can support several causes without making separate payments.	<pre> classDiagram class SplitDonationPage { -totalAmount: Float -splitAllocations: Map -selectedCampaigns: List +addCampaignToSplit() +adjustAllocation(id, amt) +showSplitSummary() +confirmSplitCheckout() +validateAllocations() } class SplitDonationController { -doneeId: String -totalAmount: Float +validateTotal() +createSplitRecord() +processPayment() +distributeToCampaigns() +generateDonations() +confirmAllDeposited() } class DonationSplit { -splitId: String -doneeId: String -totalAmount: Float -allocations: Map -createdAt: DateTime +save() / +validate() } class Donation { -donationId: String -splitId: String -campaignId: String -allocatedAmt: Float +save() / +getBySplit() } SplitDonationPage --> SplitDonationController : uses SplitDonationController --> DonationSplit : creates DonationSplit --> Donation : </pre>
6.	As a Donee, I want to view a personal impact dashboard showing total amount donated, number of campaigns funded, and how many reached their goal because of contributions I made, so that I understand my cumulative giving impact.	<pre> classDiagram class ImpactDashboardPage { -doneeId: String -summaryData: ImpactSummary +renderTotalDonated() +renderCampaignsFunded() +renderGoalsAchieved() +showImpactTimeline() +exportImpactReport() } class ImpactSummaryController { -doneeId: String +aggregateDonations() +countCampaignsFunded() +countGoalsReached() +buildImpactSummary() +getTimeline(): List +cacheSnapshot() } class DoneeImpactSummary { -doneeId: String -totalDonated: Float -campaignsFunded: Integer -goalsReached: Integer -lastUpdated: DateTime } ImpactDashboardPage --> ImpactSummaryController : uses ImpactSummaryController --> DoneeImpactSummary : </pre>
7.	As a Donee, I want to flag a campaign as suspicious and submit a reason so that the platform can investigate potential fraud and protect other donees.	<pre> classDiagram class FlagCampaignPanel { -campaignId: String -reasonCategory: String -detailText: String +openFlagDialog() +selectReasonCategory() +submitFlagReport() +confirmSubmission() +showFlagStatus() } class FraudFlagController { -doneeId: String -campaignId: String +validateFlagInput() +createFraudReport() +checkDuplicateFlag() +notifyPlatformAdmin() +updateCampaignFlag() +getFlagStatus(): String } class FraudReport { -reportId: String -doneeId: String -campaignId: String -reasonCategory: String -status: String +save() / +getStatus() } class Campaign { -campaignId: String -isFlagged: Boolean -flagCount: Integer +setFlagged() } FlagCampaignPanel --> FraudFlagController : uses FraudFlagController --> FraudReport : creates FraudReport --> Campaign : </pre>
8.	As a Donee, I want to view a live milestone feed of a campaign I donated to (e.g. "50% reached", "FR posted an update", "Goal achieved") so that I can stay engaged with the campaign's progress.	<pre> classDiagram class MilestoneFeedPage { -doneeId: String -feedItems: List +loadFeed() +displayMilestoneEvent() +showFRUpdatePost() +markEventRead() +refreshFeed() } class MilestoneFeedController { -doneeId: String +getDonatedCampaigns() +fetchMilestones() +detectNewMilestones() +buildFeedItems() +sortByRecency() +markAsRead(eventId) } class MilestoneEvent { -eventId: String -campaignId: String -eventType: String -description: String -occurredAt: DateTime +getByCampaign(): List } class Campaign { -campaignId: String -raisedAmount: Float -targetAmount: Float -status: String +getProgress(): Float } MilestoneFeedPage --> MilestoneFeedController : uses MilestoneFeedController --> MilestoneEvent : reads MilestoneFeedController --> Campaign : reads </pre>
9.	As a Donee, I want to post a public message of support on a campaign page so that I can encourage other donees and show solidarity with the cause.	<pre> classDiagram class SupportMessagePanel { -campaignId: String -messageText: String -isAnonymous: Boolean +openMessageBox() +toggleAnonymous() +submitMessage() +displayMessageFeed() +deleteOwnMessage(id) } class SupportMessageController { -doneeId: String -campaignId: String +validateMessage() +moderateContent() +postMessage() +getMessages(): List +deleteMessage(msgId) +verifyOwnership() } class SupportMessage { -messageId: String -doneeId: String -campaignId: String -content: String -isAnonymous: Boolean +save() / +delete() } class Campaign { -campaignId: String -allowMessages: Boolean +getMessages(): List } SupportMessagePanel --> SupportMessageController : uses SupportMessageController --> SupportMessage : creates SupportMessageController --> Campaign : checks </pre>

10.	As a Donee, I want to request a refund for a donation made to a campaign that was removed or flagged so that I am protected if a campaign is found to be illegitimate.	
-----	---	--

User Stories as a Fund Raiser

1.	As a Fund Raiser, I want to create a fundraising campaign with a target amount, deadline, and category so that I can clearly define my fundraising goals and attract the right donors.	
2.	As a Fund Raiser, I want to upload images and detailed descriptions for my campaign so that donors can better understand the purpose and feel more confident contributing.	
3.	As a Fund Raiser, I want to edit or update my campaign details (e.g. description, goal, deadline) while it is active so that I can keep information accurate and relevant.	

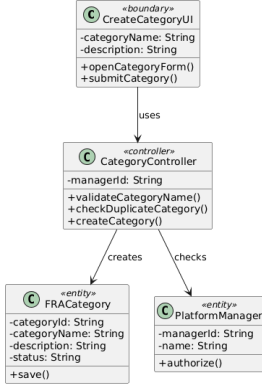
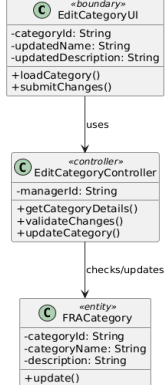
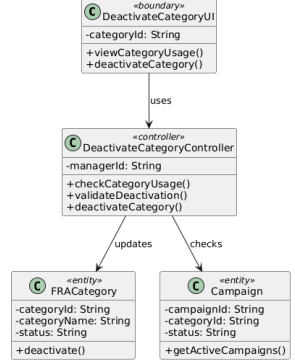
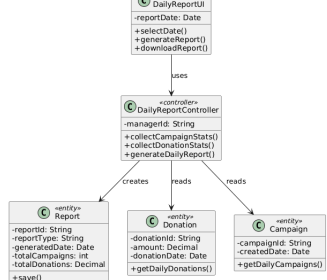
4.	As a Fund Raiser, I want to track real-time donation progress (total amount raised, % completed) so that I can monitor how close I am to reaching my goal.	<pre> classDiagram class CampaignProgressPage { <<boundary>> -campaignId: String -progressPercent: Float +viewTotalRaised() +viewPercentComplete() +refreshProgress() } class CampaignProgressController { <<controller>> -campaignId: String -totalAmount: Float +calculateTotalRaised() +calculateProgressPercentage() +fetchRecentDonations() } class Campaign { <<entity>> -campaignId: String -targetAmount: Float -currentAmount: Float +getProgress() } class Donation { <<entity>> -donationId: String -campaignId: String -amount: Float -donatedAt: DateTime +getByDonation() +sumByCampaign() } CampaignProgressPage --> CampaignProgressController : uses CampaignProgressController --> Campaign : reads CampaignProgressController --> Donation : reads </pre>
5.	As a Fund Raiser, I want to view analytics such as number of views, favourites, and donor engagement so that I can understand how my campaign is performing.	<pre> classDiagram class CampaignAnalyticsPage { <<boundary>> -campaignId: String -dateRange: DateRange +viewTotalViews() +viewFavourites() +viewDonorEngagement() +filterByDate() } class CampaignAnalyticsController { <<controller>> -campaignId: String -reportPeriod: String +countViews() +countFavourites() +calculateEngagementRate() +generateReport() } class CampaignAnalytics { <<entity>> -analyticsId: String -campaignId: String -totalViews: Integer -favourites: Integer -engagementRate: Float -reportDate: Date +save() +getLatest() } class Donation { <<entity>> -donationId: String -campaignId: String -amount: Float -donorName: String +getLatest() } CampaignAnalyticsPage --> CampaignAnalyticsController : uses CampaignAnalyticsController --> CampaignAnalytics : queries CampaignAnalyticsController --> Donation : creates </pre>
6.	As a Fund Raiser, I want to receive notifications when someone donates to my campaign so that I can stay updated and respond or acknowledge contributions promptly.	<pre> classDiagram class NotificationCenterPage { <<boundary>> -fundraiserId: String -unreadCount: Int +viewNotifications() +markAsRead() +acknowledgeContribution() } class DonationNotificationController { <<controller>> -fundraiserId: String -totalAmount: Float +detectNewDonation() +generateNotification() +sendAlert() +markNotificationRead() } class Notification { <<entity>> -notificationId: String -fundraiserId: String -message: String -isRead: Boolean -createdAt: DateTime +save() +markRead() } class Donation { <<entity>> -donationId: String -campaignId: String -amount: Float -donorName: String +getLatest() } NotificationCenterPage --> DonationNotificationController : uses DonationNotificationController --> Notification : creates DonationNotificationController --> Donation : creates </pre>
7.	As a Fund Raiser, I want to post updates (e.g. milestones, progress, news) to my campaign so that I can keep donors engaged and encourage further contributions.	<pre> classDiagram class CampaignUpdatePage { <<boundary>> -campaignId: String -updateDraft: String -updateType: String +writeUpdate() +selectUpdateType() +previewUpdate() +publishUpdate() } class CampaignUpdateController { <<controller>> -authorId: String -campaignId: String +validateContent() +createUpdate() +notifyFollowers() } class CampaignUpdate { <<entity>> -updateId: String -campaignId: String -updateType: String -content: String -postedAt: DateTime +save() +getAll() } class Donation { <<entity>> -donationId: String -campaignId: String -amount: Float -donorName: String +getLatest() } CampaignUpdatePage --> CampaignUpdateController : uses CampaignUpdateController --> CampaignUpdate : creates </pre>

8.	As a Fund Raiser, I want to withdraw collected funds once the campaign is completed or approved so that I can use the funds for the intended purpose.	<pre> classDiagram class FundWithdrawalPage { <<boundary>> -campaignId: String -bankDetails: BankInfo -withdrawAmount: Float +viewAvailableBalance() +enterBankDetails() +requestWithdrawal() +confirmWithdrawal() } class FundWithdrawalController { <<controller>> -fundraiserId: String -totalAmount: Float +validateEligibility() +verifyCampaignStatus() +processWithdrawal() +notifyFundraiser() } class WithdrawalRequest { <<entity>> -requestId: String -fundraiserId: String -amount: Float -status: String -requestedAt: DateTime +save() +getStatus() } class Campaign { <<entity>> -campaignId: String -currentAmount: Float -status: String +deductFunds() +checkStatus() } FundWithdrawalPage --> FundWithdrawalController : uses FundWithdrawalController ..> WithdrawalRequest : processes FundWithdrawalController ..> Campaign : processes </pre>
9.	As a Fund Raiser, I want to view a history of my past campaigns and their outcomes so that I can evaluate my performance and improve future campaigns.	<pre> classDiagram class CampaignHistoryPage { <<boundary>> -fundraiserId: String -filterStatus: String +viewHistory() +filterByStatus() +viewCampaignDetail() +compareCampaigns() } class CampaignHistoryController { <<controller>> -fundraiserId: String -pageSize: Int +fetchPastCampaigns() +summariseOutcomes() +applyFilter() } class Campaign { <<entity>> -campaignId: String -title: String -targetAmount: Float -finalAmount: Float -status: String -endedAt: DateTime +getByFundraiser() +getSummary() } CampaignHistoryPage --> CampaignHistoryController : uses CampaignHistoryController ..> Campaign : fetches </pre>
10.	As a Fund Raiser, I want to respond to donor comments or questions on my campaign so that I can build trust and provide transparency.	<pre> classDiagram class CampaignCommentPanel { <<boundary>> -campaignId: String -replyDraft: String +viewComments() +viewDonorQuestion() +replyToComment() +deleteReply() } class CommentResponseController { <<controller>> -fundraiserId: String -campaignId: String +loadComments() +validateReply() +saveReply() +notifyCommenter() } class Comment { <<entity>> -commentId: String -campaignId: String -donorId: String -content: String -postedAt: DateTime +getAll() +getById() } class Reply { <<entity>> -replyId: String -campaignId: String -commentId: String -fundraiserId: String -replyText: String -repliedAt: DateTime +save() +delete() } CampaignCommentPanel --> CommentResponseController : uses CommentResponseController ..> Comment : creates CommentResponseController ..> Reply : creates </pre>

User Stories as a Platform Management

1.	As a platform manager, I want to review newly submitted fundraising activities so that only legitimate campaigns are published on the platform.	<pre> classDiagram class ReviewSubmissionUI { <<boundary>> -managerId: String -submissionFilter: String +viewSubmittedCampaigns() +openCampaignDetails() } class ReviewSubmissionController { <<controller>> -managerId: String +getSubmittedCampaigns() +loadSubmittedCampaignDetails(campaignId) +validateManagerAccess() } class PlatformManager { <<entity>> -managerId: String -name: String -email: String +getRole() } class Campaign { <<entity>> -campaignId: String -title: String -status: String -submittedDate: Date +getDetails() } ReviewSubmissionUI --> ReviewSubmissionController : uses ReviewSubmissionController ..> PlatformManager : checks ReviewSubmissionController ..> Campaign : retrieves </pre>
----	---	--

2.	As a platform manager, I want to approve a fundraising activity with a published status so that donees can start viewing and donating to it.	<pre> classDiagram class ApproveCampaignUI { <<boundary>> -campaignId: String -approvalNote: String +openApprovalPage() +submitApproval() } class ApproveCampaignController { <<controller>> -managerId: String +validateCampaign() +approveCampaign(campaignId) +recordApprovalNote() +notifyFundRaiser() } class Campaign { <<entity>> -campaignId: String -title: String -status: String -approvedDate: Date +approve() } class ApprovalRecord { <<entity>> -approvalId: String -campaignId: String -managerId: String -note: String +save() } ApproveCampaignUI --> ApproveCampaignController : uses ApproveCampaignController --> Campaign : updates ApproveCampaignController --> ApprovalRecord : creates </pre>
3.	As a platform manager, I want to reject a fundraising activity with remarks so that the fund raiser knows what must be corrected before resubmission.	<pre> classDiagram class RejectCampaignUI { <<boundary>> -campaignId: String -rejectionReason: String +openRejectionForm() +submitRejection() } class RejectCampaignController { <<controller>> -managerId: String +validateRejectionReason() +rejectCampaign(campaignId) +saveRemarks() +notifyFundRaiser() } class Campaign { <<entity>> -campaignId: String -title: String -status: String +reject() } class RejectionRecord { <<entity>> -rejectionId: String -campaignId: String -remarks: String -rejectedDate: Date +save() } RejectCampaignUI --> RejectCampaignController : uses RejectCampaignController --> Campaign : updates RejectCampaignController --> RejectionRecord : creates </pre>
4.	As a platform manager, I want to suspend a fundraising activity that violates platform rules so that harmful or misleading campaigns are removed from public view.	<pre> classDiagram class SuspendCampaignUI { <<boundary>> -campaignId: String -suspensionReason: String +viewCampaign() +submitSuspension() } class SuspendCampaignController { <<controller>> -managerId: String +checkViolation() +suspendCampaign(campaignId) +logSuspension() +notifyFundRaiser() } class Campaign { <<entity>> -campaignId: String -status: String -isVisible: Boolean +suspend() } class SuspensionRecord { <<entity>> -recordId: String -campaignId: String -reason: String -suspendedDate: Date +save() } SuspendCampaignUI --> SuspendCampaignController : uses SuspendCampaignController --> Campaign : updates SuspendCampaignController --> SuspensionRecord : creates </pre>

5.	As a platform manager, I want to create new fundraising categories so that campaigns can be organized properly for searching and reporting.	 <pre> classDiagram class CreateCategoryUI { <<boundary>> -categoryName: String -description: String +openCategoryForm() +submitCategory() } class CategoryController { <<controller>> -managerId: String +validateCategoryName() +checkDuplicateCategory() +createCategory() } class FRACategory { <<entity>> -categoryId: String -categoryName: String -description: String -status: String +save() } class PlatformManager { <<entity>> -managerId: String -name: String +authorize() } CreateCategoryUI --> CategoryController : uses CategoryController --> FRACategory : creates CategoryController --> PlatformManager : checks </pre>
6.	As a platform manager, I want to edit category names and descriptions so that the platform classification stays accurate and up to date.	 <pre> classDiagram class EditCategoryUI { <<boundary>> -categoryId: String -updatedName: String -updatedDescription: String +loadCategory() +submitChanges() } class EditCategoryController { <<controller>> -managerId: String +getCategoryDetails() +validateChanges() +updateCategory() } class FRACategory { <<entity>> -categoryId: String -categoryName: String -description: String +update() } EditCategoryUI --> EditCategoryController : uses EditCategoryController --> FRACategory : checks/updates </pre>
7.	As a platform manager, I want to deactivate unused or invalid categories so that fund raisers cannot assign campaigns to outdated categories.	 <pre> classDiagram class DeactivateCategoryUI { <<boundary>> -categoryId: String +viewCategoryUsage() +deactivateCategory() } class DeactivateCategoryController { <<controller>> -managerId: String +checkCategoryUsage() +validateDeactivation() +deactivateCategory() } class FRACategory { <<entity>> -categoryId: String -categoryName: String -status: String +deactivate() } class Campaign { <<entity>> -campaignId: String -categoryId: String -status: String +getActiveCampaigns() } DeactivateCategoryUI --> DeactivateCategoryController : uses DeactivateCategoryController --> FRACategory : updates DeactivateCategoryController --> Campaign : checks </pre>
8.	As a platform manager, I want to generate daily reports of fundraising activity and donations so that I can monitor short-term platform performance.	 <pre> classDiagram class DailyReportUI { <<boundary>> -reportDate: Date +selectDate() +generateReport() +downloadReport() } class DailyReportController { <<controller>> -managerId: String +collectCampaignStats() +collectDonationStats() +generateDailyReport() } class Report { <<entity>> -reportId: String -reportType: String -generatedDate: Date -totalCampaigns: int -totalDonations: Decimal +save() } class Donation { <<entity>> -donationId: String -amount: Decimal -donationDate: Date +getDailyDonations() } class Campaign { <<entity>> -campaignId: String -categoryId: String -createdDate: Date +getDailyCampaigns() } DailyReportUI --> DailyReportController : uses DailyReportController --> Report : creates DailyReportController --> Donation : reads DailyReportController --> Campaign : reads </pre>

9.	As a platform manager, I want to generate weekly and monthly reports by category so that I can identify trends in campaign creation and donor activity.	<pre> classDiagram class SummaryReportUI { <<boundary>> -periodType: String -selectedCategory: String +choosePeriod() +filterByCategory() +generateSummaryReport() } class SummaryReportController { <<controller>> -managerId: String +getCampaignTotals() +getDonationTotals() +groupByCategory() +generateReport() } class Report { <<entity>> -reportId: String -periodType: String -generatedDate: Date +save() } class FRACategory { <<entity>> -categoryId: String -categoryName: String +getCategoryName() } class Donation { <<entity>> -donationId: String -amount: Decimal +getTotals() } SummaryReportUI --> SummaryReportController : uses SummaryReportController --> Report : creates SummaryReportController --> FRACategory : filters SummaryReportController --> Donation : reads </pre>
10.	As a platform manager, I want to search and view the history of approved, rejected and suspended fundraising activities so that I can audit past management decisions.	<pre> classDiagram class CampaignHistoryUI { <<boundary>> -statusFilter: String -dateFrom: Date -dateTo: Date +searchHistory() +viewHistoryDetails() } class CampaignHistoryController { <<controller>> -managerId: String +filterByStatus() +filterByDate() +retrieveHistory() } class Campaign { <<entity>> -campaignId: String -title: String -status: String -submittedDate: Date +getHistory() } class ManagementAction { <<entity>> -actionId: String -campaignId: String -actionType: String -actionDate: Date -remarks: String +save() } CampaignHistoryUI --> CampaignHistoryController : uses CampaignHistoryController --> Campaign : checks CampaignHistoryController --> ManagementAction : retrieves </pre>

User Stories as an Insurance Assessor

1.	As an Assessor, I want to view all claims assigned to my queue so that I can manage my workload and prioritise which claims to assess next.	<pre> classDiagram class AssessorQueuePage { <<boundary>> -assessorId: String -claimList: List -filterStatus: String +loadQueue() +displayClaims() +sortByPriority() +filterByStatus() } class ClaimQueueController { <<controller>> -assessorId: String -queue: List +getQueue(): List +sortClaims() +filterClaims() +getClaimDetails() } class Claim { <<entity>> -claimId: String -policyId: String -status: ClaimStatus -lodgedDate: Date -priority: Integer +getStatus() +save() } AssessorQueuePage --> ClaimQueueController : uses ClaimQueueController --> Claim : reads </pre>
2.	As an Assessor, I want to record interview notes (with police, drivers, or witnesses) against a claim so that all gathered evidence is tracked in the system for audit purposes.	<pre> classDiagram class InterviewNotesForm { <<boundary>> -claimId: String -intervieweeType: String -noteText: String +displayForm() +submitNote() +validateInput() } class InterviewNoteCtrl { <<controller>> -claimId: String -assessorId: String +saveNote(note) +getNotesByClaim() +linkInterviewee() +getInterviewRecord() } class InterviewRecord { <<entity>> -recordId: String -claimId: String -intervieweeType: String -notes: String -dateRecorded: DateTime +save() +getRecord(): Record } InterviewNotesForm --> InterviewNoteCtrl : uses InterviewNoteCtrl --> InterviewRecord : writes </pre>

3.	As an Assessor, I want to approve or reject a claim and record my decision with justification so that the outcome is formally documented and communicated to the policyholder.	«boundary» ClaimDecisionPage - claimId: String - decision: String - justification: String + displayClaimDetails() + submitDecision() + showConfirmDialog() + notifyPolicyholder() «controller» ClaimDecisionCtrl - claimId: String + approveClaim() + rejectClaim() + recordJustification() + updatePremium() + notifyPolicyholder() + adjustDriverRating() «entity» ClaimDecision - decisionId: String - claimId: String - outcome: String - justification: String - decidedAt: DateTime - assessorId: String + save() + getDecision()
4.	As an Assessor, I want to flag a claim as potentially fraudulent so that it can be escalated to the investigations branch for further review.	«boundary» FraudFlagPanel - claimId: String - flagReason: String - suspicionLevel: String + showFlagForm() + submitFlag() + confirmEscalation() «controller» FraudEscalationCtrl - claimId: String + flagAsFraud() + escalateToBranch() + notifyInvestigations() + recordFlagReason() + getFraudHistory() «entity» FraudFlag - flagId: String - claimId: String - reason: String - suspicionLevel: String - flaggedAt: DateTime + save() + escalate()
5.	As an Assessor, I want to upload photos and inspection notes after a physical vehicle inspection so that visual evidence is attached directly to the damage claim record.	«boundary» InspectionUploadPage - claimId: String - photoFiles: List - inspectionNotes: String + selectFiles() + uploadPhotos() + submitNotes() + showUploadProgress() «controller» InspectionCtrl - claimId: String + saveInspection() + attachPhotos() + getInspections() + validateFiles() + linkToClaim() «entity» InspectionReport - reportId: String - claimId: String - photoUrls: List - notes: String - inspectedAt: DateTime + save() + getPhotos(): List
6.	As an Assessor, I want to verify that a police report has been received for stolen vehicle or malicious damage claims so that I do not proceed with assessment before mandatory documentation is in place.	«boundary» PoliceReportCheckPage - claimId: String - reportStatus: String + displayReportStatus() + markReportReceived() + blockAssessment() + sendReminder() «controller» PoliceReportCtrl - claimId: String + checkReportStatus() + confirmReceipt() + blockIfMissing() + getPoliceReport() + sendChaseAlert() «entity» PoliceReport - reportId: String - claimId: String - isReceived: Boolean - receivedDate: Date - reportNumber: String + confirmReceipt() + save()
7.	As an Assessor, I want to view the full policy details of the claimant (coverage type, excess, inclusions, driver rating) so that I can make an informed decision on the claim payout.	«boundary» PolicyDetailPage - policyId: String - coverageSummary: String + displayPolicyInfo() + showDriverRating() + showExcessAmount() + showClaimHistory() «controller» PolicyLookupCtrl - policyId: String + getPolicy(): Policy + getDriverProfile() + computeExcess() + getCoverageScope() + getPastClaims() «entity» Policy - policyId: String - coverageType: String - excess: Float - driverRating: Integer - inclusions: List + getExcess() + getCoverage()
8.	As an Assessor, I want to calculate and confirm the payout amount (insured value minus excess) for an approved claim so that the correct amount is disbursed electronically to the policyholder.	«boundary» PayoutCalculatorPage - claimId: String - insuredValue: Float - excessAmount: Float + displayCalculation() + confirmPayout() + triggerDisbursement() «controller» PayoutCtrl - claimId: String + computePayout() + applyExcessDeduction() + initiateDisbursement() + recordTransaction() + notifyPolicyholder() «entity» PayoutRecord - payoutId: String - claimId: String - grossAmount: Float - excessDeducted: Float - netPayout: Float + compute() + save()

9.	As an Assessor, I want to update a policyholder's driver rating after an at-fault accident claim is finalised so that their premium is correctly adjusted for the next renewal period.	<pre> classDiagram class DriverRatingUpdatePage { <<boundary>> - policyId: String - currentRating: Integer - newRating: Integer + displayCurrentRating() + confirmRatingChange() + showPremiumImpact() } class DriverRatingCtrl { <<controller>> - policyId: String + decrementRating() + recomputePremium() + removeNoClaim() + saveRatingChange() + notifyPolicyholder() } class DriverProfile { <<entity>> - profileId: String - policyId: String - rating: Integer - noClaimBonus: Boolean - atFaultCount: Integer + updateRating() + save() } DriverRatingUpdatePage --> DriverRatingCtrl : uses DriverRatingCtrl --> DriverProfile : writes </pre>
10.	As an Assessor, I want to re-open a stolen vehicle claim as a damage claim when a previously stolen car is recovered damaged so that the assessment can continue under the correct claim type.	<pre> classDiagram class ClaimConversionPage { <<boundary>> - claimId: String - recoveryDate: Date - damageDescription: String + showConversionForm() + confirmConversion() + displayNewClaimType() } class ClaimConversionCtrl { <<controller>> - claimId: String + convertClaimType() + reOpenClaim() + linkRecoveryRecord() + updateClaimStatus() + notifyPolicyholder() } class Claim { <<entity>> - claimId: String - type: ClaimType - status: ClaimStatus - recoveryDate: Date - damageNotes: String + convertType() + save() } ClaimConversionPage --> ClaimConversionCtrl : uses ClaimConversionCtrl --> Claim : writes </pre>

User stories as a **System Administrator**

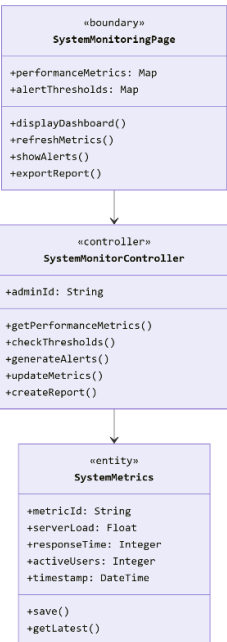
As a System Admin, I want to create new user accounts for Fund Raisers, Donees, and Platform Managers so that new users can access the platform.	<pre> classDiagram class CreateUserPage { <<boundary>> +username: String +email: String +role: String +displayForm() +validateInput() +submitForm() +showConfirmation() } class UserManagementController { <<controller>> +adminId: String +createUser() +validateUserData() +checkDuplicateUsername() +assignDefaultRole() +sendWelcomeEmail() } class User { <<entity>> +userId: String +username: String +email: String +role: String +status: String +createdAt: DateTime +save() +validate() } CreateUserPage --> UserManagementController UserManagementController --> User </pre>
As a System Admin, I want to view and search all user accounts by username, email, or role so that I can quickly locate specific users for management purposes.	
As a System Admin, I want to suspend user accounts that violate platform policies so that harmful users cannot access the system.	
As a System Admin, I want to reactivate suspended accounts after issues are resolved so that legitimate users can regain access.	
As a System Admin, I want to delete inactive or fraudulent accounts permanently so that the user database remains clean and secure.	
As a System Admin, I want to reset user passwords and send secure temporary passwords so that users can regain access to locked accounts.	

As a System Admin, I want to monitor system performance metrics (server load, response times, active users) so that I can ensure optimal platform performance.

As a System Admin, I want to view real-time platform statistics (total campaigns, donations, user activity) so that I can track overall system health.

As a System Admin, I want to manage platform configuration settings (donation limits, campaign duration limits, fee rates) so that business rules can be updated as needed.

As a System Admin, I want to backup and restore system data regularly so that critical information is protected against data loss.



Donor

1	As a Donor, I want to register an account, so that I can make donations and track my history.	<pre> classDiagram class RegisterPage { +showForm() +getInputs() +showConfirmation() } class RegisterController { +validateInputs() +checkDuplicate() +createDonor() } class Donor { -donorId : String -email : String +save() } RegisterPage --> RegisterController RegisterController --> Donor </pre>
2	As a Donor, I want to browse active fundraising campaigns, so that I can find causes I want to support.	<pre> classDiagram class BrowseCampaignPage { +displayCampaigns() +showCategory() +selectCampaign() } class BrowseCampaignController { +getActiveCampaigns() +sortByDate() +sortByGoal() } class Campaign { -campaignId : String -title : String +getActive() } BrowseCampaignPage --> BrowseCampaignController BrowseCampaignController --> Campaign </pre>
3	As a Donor, I want to search campaigns by category or keyword, so that I can quickly find relevant causes.	<pre> classDiagram class BrowseCampaignPage { +showSearchBar() +displayResults() } class SearchCampaignController { +searchByKeyword() +filterByCategory() +returnResults() } class Campaign { -category : String -keywords : String +search(query) } BrowseCampaignPage --> SearchCampaignController SearchCampaignController --> Campaign </pre>
4	As a Donor, I want to view the details of a campaign, so that I can decide whether to donate.	<pre> classDiagram class CampaignDetailPage { +showDetails() +showFundRaiserInfo() +showGoalProgress() } class ViewCampaignController { +getCampaignById() +getFundRaiserInfo() +getProgress() } class Campaign { -description : String -goalAmount : float +getById(id) } CampaignDetailPage --> ViewCampaignController ViewCampaignController --> Campaign </pre>
5	As a Donor, I want to make a donation to a campaign, so that I can contribute to a cause.	<pre> classDiagram class DonatePage { +showDonateForm() +getAmount() +submitDonation() } class DonateController { +validateAmount() +processDonation() +updateCampaign() } class Donation { -amount : float +save() } class Campaign { +updateRaised() } DonatePage --> DonateController DonateController --> Donation DonateController --> Campaign </pre>
6	As a Donor, I want to receive a confirmation after donating, so that I have proof of my contribution.	<pre> classDiagram class DonatePage { +showReceipt() +showThankYou() +emailConfirmation() } class ConfirmDonationController { +generateReceipt() +sendEmailReceipt() +logConfirmation() } class Donation { -confirmationNo : String -date : Date +getReceipt() } DonatePage --> ConfirmDonationController ConfirmDonationController --> Donation </pre>
7	As a Donor, I want to view my donation history, so that I can keep track of all my past donations.	<pre> classDiagram class DonationHistoryPage { +showHistory() +showDonationRow() +showTotalDonated() } class DonationHistoryController { +getDonorHistory() +calculateTotal() +sortByDate() } class Donation { -donorId : String -campaignId : String +getByDonor(id) } DonationHistoryPage --> DonationHistoryController DonationHistoryController --> Donation </pre>
8	As a Donor, I want to save campaigns to a watchlist, so that I can follow their progress over time.	<pre> classDiagram class CampaignDetailPage { +showSaveButton() +showSavedState() } class SaveWatchlistController { +addToWatchlist() +checkIfSaved() +confirmSaved() } class Watchlist { -donorId : String -campaignIds : List +add(campaignId) } CampaignDetailPage --> SaveWatchlistController SaveWatchlistController --> Watchlist </pre>
9	As a Donor, I want to filter my donation history by date or category, so that I can find specific past donations easily.	<pre> classDiagram class DonationHistoryPage { +showFilterPanel() +displayFiltered() } class FilterDonationController { +filterByDate() +filterByCategory() +returnFiltered() } class Donation { -date : Date -category : String +filterBy(params) } DonationHistoryPage --> FilterDonationController FilterDonationController --> Donation </pre>
10	As a Donor, I want to see the progress of a campaign I donated to, so that I know how close it is to its goal.	<pre> classDiagram class CampaignDetailPage { +showProgressBar() +showRaisedAmount() } class ViewProgressController { +getCampaignProgress() +calcPercentage() +getDonorContrib() } class Campaign { +getProgress() } class Donation { +getByDonorCampaign() } CampaignDetailPage --> ViewProgressController ViewProgressController --> Campaign ViewProgressController --> Donation </pre>

Compliance

1	As a compliance officer, I want to monitor regulatory updates so that I can ensure company policies remain compliant.	<pre> classDiagram class RegulatoryDashboard { <<boundary>> +showUpdates() +alertChanges() +alertChanges() } class RegulatoryUpdateController { <<control>> +fetchUpdates() +analyzeImpact() +notifyChanges() } class Regulation { <<entity>> +id: String +description: String } class Policy { <<entity>> +id: String +status: String } RegulatoryDashboard --> RegulatoryUpdateController RegulatoryUpdateController --> Regulation RegulatoryUpdateController --> Policy </pre>
2	As a compliance officer, I want to assess compliance risks so that I can prioritize mitigation actions.	<pre> classDiagram class RiskAssessmentUI { <<boundary>> +inputRiskData() +viewAssessment() } class RiskAssessmentController { <<control>> +evaluateRisk() +generateReport() } class Risk { <<entity>> +level: String +impact: String } class AssessmentReport { <<entity>> +summary: String } RiskAssessmentUI --> RiskAssessmentController RiskAssessmentController --> Risk RiskAssessmentController --> AssessmentReport </pre>
3	As a compliance officer, I want to create and update policies so that employees follow current regulations.	<pre> classDiagram class PolicyManagementUI { <<boundary>> +createPolicy() +updatePolicy() } class PolicyController { <<control>> +validatePolicy() +savePolicy() } class Policy { <<entity>> +name: String +version: String } class Regulation { <<entity>> +id: String } PolicyManagementUI --> PolicyController PolicyController --> Policy PolicyController --> Regulation </pre>
4	As a compliance officer, I want to track audit trails so that I can ensure accountability and transparency.	<pre> classDiagram class AuditTrailViewer { <<boundary>> +viewLogs() } class AuditController { <<control>> +fetchLogs() +filterLogs() } class AuditLog { <<entity>> +timestamp: Date } class UserActivity { <<entity>> +action: String } AuditTrailViewer --> AuditController AuditController --> AuditLog AuditController --> UserActivity </pre>
5	As a compliance officer, I want to log and track incidents so that I can investigate compliance breaches.	<pre> classDiagram class IncidentReportingUI { <<boundary>> +reportIncident() } class IncidentController { <<control>> +logIncident() +assignInvestigation() } class Incident { <<entity>> +severity: String } class Investigation { <<entity>> +status: String } IncidentReportingUI --> IncidentController IncidentController --> Incident IncidentController --> Investigation </pre>
6	As a compliance officer, I want to track employee training completion so that I can ensure regulatory training requirements are met.	<pre> classDiagram class TrainingDashboard { <<boundary>> +viewTrainingStatus() } class TrainingController { <<control>> +trackCompletion() } class TrainingRecord { <<entity>> +completionDate: Date } class Employee { <<entity>> +name: String } TrainingDashboard --> TrainingController TrainingController --> TrainingRecord TrainingController --> Employee </pre>
7	As a compliance officer, I want to manage compliance documents so that I can maintain proper records.	<pre> classDiagram class DocumentManagementUI { <<boundary>> +uploadDocument() +viewDocument() } class DocumentController { <<control>> +saveDocument() +versionControl() } class Document { <<entity>> +title: String } class Version { <<entity>> +number: int } DocumentManagementUI --> DocumentController DocumentController --> Document DocumentController --> Version </pre>

8	As a compliance officer, I want to evaluate third-party vendors so that I can ensure they meet compliance standards.	UML diagram showing the Vendor Assessment UI. The diagram includes a boundary component 'VendorAssessmentUI' with a method 'assessVendor()'. This component interacts with a control component 'VendorController' which has a method 'evaluateCompliance()'. The 'VendorController' interacts with two entity components: 'Vendor' (with attribute 'name: String') and 'ComplianceScore' (with attribute 'score: Int').
9	As a compliance officer, I want to generate compliance reports so that I can present insights to management.	UML diagram showing the Reporting Dashboard. The diagram includes a boundary component 'ReportingDashboard' with a method 'viewReports()'. This component interacts with a control component 'ReportingController' which has a method 'generateReport()'. The 'ReportingController' interacts with two entity components: 'Report' (with attribute 'title: String') and 'Metrics' (with attribute 'value: float').
10	As a compliance officer, I want to submit regulatory filings so that I can meet legal reporting requirements.	UML diagram showing the Filing Submission UI. The diagram includes a boundary component 'FilingSubmissionUI' with a method 'submitFiling()'. This component interacts with a control component 'FilingController' which has methods 'validateFiling()' and 'sendFiling()'. The 'FilingController' interacts with two entity components: 'Filing' (with attribute 'status: String') and 'Regulation' (with attribute 'id: String').